

7 - Complexity

May 19, 2026

Độ phức tạp của thuật toán (Complexity)

TS. Lê Thành Trung

May 19, 2026

Contents

1	Mục tiêu bài học	3
2	Độ phức tạp và ký hiệu Big-O	3
2.1	Độ phức tạp là gì?	3
2.2	Ký hiệu Big-O	3
2.3	Cách ước lượng Big-O	3
3	Vì sao độ phức tạp quan trọng?	5
3.1	Lý do thực tế	5
3.2	Ví dụ trực quan	5
4	Bộ phân tích hiệu năng (The Profiler)	5
4.1	Dùng magic command	5
4.2	Dùng Python Profiler	6
5	Tổng kết	9
6	Bài tập (Problems)	9
6.1	Bài 1: Nhận diện độ phức tạp	9
6.2	Bài 2: Tính Big-O từ mã nguồn	10
6.3	Bài 3: So sánh bằng magic command	10
6.4	Bài 4: Dùng Python profiler trong notebook	10
6.5	Bài 5: So sánh thời gian khi dùng và không dùng list comprehension	10
6.6	Bài 6: Vận dụng thực tế	10

1 Mục tiêu bài học

- Hiểu khái niệm độ phức tạp (complexity) và ký hiệu Big-O.
- Đánh giá độ phức tạp thời gian (time complexity) và bộ nhớ (space complexity) ở mức cơ bản.
- Hiểu vì sao độ phức tạp quan trọng trong thực tế.
- Biết dùng công cụ đo hiệu năng (profiler) để kiểm tra mã nguồn.

2 Độ phức tạp và ký hiệu Big-O

2.1 Độ phức tạp là gì?

Độ phức tạp mô tả mức tài nguyên thuật toán cần dùng khi kích thước dữ liệu tăng.

- Độ phức tạp thời gian (time complexity): số bước xử lý theo kích thước đầu vào n .
- Độ phức tạp bộ nhớ (space complexity): lượng bộ nhớ bổ sung cần dùng theo n .

2.2 Ký hiệu Big-O

Big-O diễn tả tốc độ tăng của chi phí xử lý khi n lớn.

Một số mức thường gặp: - $O(1)$: hằng số (constant) - $O(\log n)$: logarit (logarithmic) - $O(n)$: tuyến tính (linear) - $O(n \log n)$: tuyến tính nhân logarit - $O(n^2)$: bậc hai (quadratic) - $O(2^n)$: lũy thừa (exponential)

2.3 Cách ước lượng Big-O

Khi tính Big-O, ta không đếm chính xác từng phép toán, mà chỉ quan tâm xu hướng tăng khi n đủ lớn.

Nguyên tắc cơ bản: - Phép gán, cộng, trừ, truy cập một phần tử: thường xem là $O(1)$. - Một vòng lặp chạy n lần: thường là $O(n)$. - Hai vòng lặp lồng nhau cùng chạy n lần: thường là $O(n^2)$. - Nếu mỗi bước giảm bài toán đi một nửa, thường là $O(\log n)$. - Nhiều đoạn mã chạy nối tiếp: lấy bậc lớn nhất. Ví dụ $O(n) + O(n^2) = O(n^2)$. - Bỏ hằng số và hệ số. Ví dụ $O(2n + 3)$ viết gọn là $O(n)$.

Ví dụ nhanh: - `for i in range(n):` $O(n)$ - `for i in range(n): for j in range(n):` $O(n^2)$ - `while n > 1: n //= 2:` $O(\log n)$

[23]: *# Ví dụ: nhận diện cách tính Big-O*

```
def constant_example(a_list):
    return a_list[0] + a_list[-1]  #  $O(1)$ 

def sequential_example(a_list):
    total = 0
    for value in a_list:  #  $O(n)$ 
        total += value
    return total
```

```

def nested_example(a_list):
    count = 0
    for _ in a_list:          #  $O(n)$ 
        for _ in a_list:      #  $O(n)$ 
            count += 1
    return count              #  $O(n^2)$ 

def logarithmic_example(n):
    steps = 0
    while n > 1:               # mỗi bước giảm một nửa
        n //= 2
        steps += 1
    return steps              #  $O(\log n)$ 

nums = [1, 2, 3, 4, 5]
print("constant_example:", constant_example(nums))
print("sequential_example:", sequential_example(nums))
print("nested_example:", nested_example(nums))
print("logarithmic_example(32):", logarithmic_example(32))

```

```

constant_example: 6
sequential_example: 15
nested_example: 25
logarithmic_example(32): 5

```

[24]: # Ví dụ nhanh: ước lượng độ phức tạp

```

def get_first(a_list):
    return a_list[0] #  $O(1)$ 

def linear_search(a_list, target):
    for x in a_list: #  $O(n)$ 
        if x == target:
            return True
    return False

def bubble_like_count(a_list):
    count = 0
    for i in range(len(a_list)): #  $O(n)$ 
        for j in range(len(a_list)): #  $O(n)$ 
            count += 1 #  $O(n^2)$  tổng thể
    return count

nums = list(range(10))

```

```
print("get_first:", get_first(nums))
print("linear_search 9:", linear_search(nums, 9))
print("nested loop count:", bubble_like_count(nums))
```

```
get_first: 0
linear_search 9: True
nested loop count: 100
```

3 Vì sao độ phức tạp quan trọng?

3.1 Lý do thực tế

- Dữ liệu lớn hơn làm thuật toán chậm đi rất nhanh nếu chọn sai cấu trúc.
- Big-O giúp dự đoán khả năng mở rộng (scalability).
- Giúp quyết định giữa nhiều cách cài đặt có cùng kết quả.

3.2 Ví dụ trực quan

Nếu $n = 10^6$: - $O(n)$ khoảng một triệu bước. - $O(n^2)$ khoảng một nghìn tỷ bước.

Khác biệt này quyết định chương trình chạy trong giây hay trong nhiều giờ.

4 Bộ phân tích hiệu năng (The Profiler)

Profiler là công cụ đo xem đoạn nào trong chương trình tốn thời gian nhất.

4.1 Dùng magic command

Ngay cả khi việc tối ưu không làm thay đổi độ phức tạp Big-O của chương trình, nhiều lập trình viên vẫn sẵn sàng dành nhiều thời gian để làm cho mã nguồn (code) chạy nhanh hơn, dù chỉ là một cải thiện nhỏ.

Trong Jupyter Notebook, có một số magic commands giúp kiểm tra thời gian chạy (run time) của mã nguồn một cách nhanh chóng:

- `%time`: đo thời gian chạy của một câu lệnh đơn (single statement).
- `%timeit`: đo thời gian chạy lặp lại của một câu lệnh đơn (repeated run time of a single statement).
- `%%time`: đo thời gian chạy của toàn bộ mã trong một ô (the run time of all code in the cell).
- `%%timeit`: đo thời gian chạy lặp lại của toàn bộ một ô (the repeated run time of a cell).

```
[10]: # Ví dụ các magic commands để đo thời gian
```

```
def sum_squares(n):
    return sum(i * i for i in range(n))
```

```
[11]: %time sum_squares(1000000)
```

```
CPU times: total: 31.2 ms
Wall time: 55.4 ms
```

```
[11]: 33332833333500000
```

```
[13]: %%timeit sum_squares(10000)
```

```
468 s ± 6.08 s per loop (mean ± std. dev. of 7 runs, 1,000 loops each)
```

```
[15]: %%time
total = sum_squares(100000)
print(total)
```

```
3333283333350000
```

```
CPU times: total: 15.6 ms
```

```
Wall time: 5 ms
```

```
[16]: %%timeit
total = sum_squares(100_000)
```

```
5.1 ms ± 44.6 s per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

4.2 Dùng Python Profiler

Bạn cũng có thể dùng Python profiler để phân tích (profile) đoạn mã mình viết. Trong Jupyter Notebook:

- `%prun`: chạy một câu lệnh đơn (single statement) thông qua Python code profiler.
- `%%prun`: chạy toàn bộ một ô (a cell) thông qua Python code profiler.

```
[17]: def work(n):
      total = 0
      for i in range(n):
          total += i * i
      return total

      def run_workload():
          for _ in range(50):
              work(20_000)
```

```
[18]: %%prun work(100_000)
```

```
4 function calls in 0.005 seconds
```

```
Ordered by: internal time
```

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.004	0.004	0.004	0.004	905316236.py:1(work)
1	0.000	0.000	0.000	0.000	{method 'disable' of
'_lsprof.Profiler' objects}					
1	0.000	0.000	0.004	0.004	{built-in method builtins.exec}
1	0.000	0.000	0.004	0.004	<string>:1(<module>)

```
[19]: %%prun
run_workload()
```

252 function calls in 0.072 seconds

Ordered by: internal time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
50	0.040	0.001	0.040	0.001	905316236.py:1(work)
1	0.024	0.024	0.031	0.031	base_events.py:1894(_run_once)
1	0.006	0.006	0.006	0.006	_base.py:337(_invoke_callbacks)
12	0.000	0.000	0.000	0.000	socket.py:626(send)
2	0.000	0.000	0.000	0.000	{method '__exit__' of
					'sqlite3.Connection' objects}
1	0.000	0.000	0.000	0.000	{method 'disable' of
					'_lsprof.Profiler' objects}
2	0.000	0.000	0.000	0.000	traitlets.py:3631(set)
1	0.000	0.000	0.006	0.006	
					kernelbase.py:302(poll_control_queue)
11	0.000	0.000	0.000	0.000	enum.py:1531(__or__)
1	0.000	0.000	0.000	0.000	
					selector_events.py:141(_write_to_self)
1	0.000	0.000	0.000	0.000	{built-in method select.select}
2	0.000	0.000	0.000	0.000	traitlets.py:718(_validate)
1	0.000	0.000	0.000	0.000	queues.py:225(get)
1	0.000	0.000	0.006	0.006	{method 'run' of
					'_contextvars.Context' objects}
1	0.000	0.000	0.000	0.000	
					base_events.py:835(call_soon_threadsafe)
1	0.000	0.000	0.000	0.000	selectors.py:319(select)
1	0.000	0.000	0.000	0.000	
					traitlets.py:1527(_notify_observers)
37	0.000	0.000	0.000	0.000	{built-in method
					builtins.isinstance}
1	0.000	0.000	0.000	0.000	selectors.py:313(_select)
1	0.000	0.000	0.000	0.000	traitlets.py:1512(_notify_trait)
1	0.000	0.000	0.000	0.000	queues.py:256(get_nowait)
1	0.000	0.000	0.000	0.000	
					iostream.py:213(_is_master_process)
11	0.000	0.000	0.000	0.000	enum.py:713(__call__)
1	0.000	0.000	0.006	0.006	_base.py:537(set_result)
2	0.000	0.000	0.000	0.000	traitlets.py:3474(validate)
1	0.000	0.000	0.012	0.012	905316236.py:8(run_workload)
2	0.000	0.000	0.000	0.000	traitlets.py:727(_cross_validate)
2	0.000	0.000	0.000	0.000	

```

traitlets.py:3624(validate_elements)
    1  0.000  0.000  0.000  0.000 futures.py:394(_call_set_state)
    2  0.000  0.000  0.000  0.000 0.000 traitlets.py:689(set)
    1  0.000  0.000  0.000  0.000 0.000 {method 'send' of '_socket.socket'
objects}
    5  0.000  0.000  0.000  0.000 0.000 {built-in method builtins.hasattr}
    1  0.000  0.000  0.000  0.000 0.000 {built-in method
_thread.allocate_lock}
    2  0.000  0.000  0.000  0.000 0.000 threading.py:299(__enter__)
    1  0.000  0.000  0.000  0.012 0.012 {built-in method builtins.exec}
   11  0.000  0.000  0.000  0.000 0.000 enum.py:1116(__new__)
    1  0.000  0.000  0.000  0.000 0.000 base_events.py:731(time)
    2  0.000  0.000  0.000  0.000 0.000 traitlets.py:708(__set__)
    1  0.000  0.000  0.000  0.000 0.000
history.py:839(_writeout_output_cache)
    2  0.000  0.000  0.000  0.000 0.000 traitlets.py:2304(validate)
    1  0.000  0.000  0.000  0.000 0.000 traitlets.py:1523(notify_change)
    1  0.000  0.000  0.000  0.000 0.000 events.py:32(__init__)
    7  0.000  0.000  0.000  0.000 0.000 {built-in method builtins.len}
    2  0.000  0.000  0.000  0.000 0.000 {method 'popleft' of
'collections.deque' objects}
    1  0.000  0.000  0.000  0.006 0.006 events.py:82(_run)
    1  0.000  0.000  0.000  0.000 0.000 base_events.py:811(_call_soon)
    2  0.000  0.000  0.000  0.000 0.000 traitlets.py:676(__get__)
    1  0.000  0.000  0.000  0.000 0.000 queues.py:173(qsize)
    1  0.000  0.000  0.000  0.000 0.000 queues.py:322(_consume_expired)
    1  0.000  0.000  0.000  0.000 0.000 {built-in method builtins.max}
    1  0.000  0.000  0.000  0.000 0.000 threading.py:424(notify_all)
    1  0.000  0.000  0.000  0.012 0.012 <string>:1(<module>)
    1  0.000  0.000  0.000  0.000 0.000 threading.py:394(notify)
    1  0.000  0.000  0.000  0.000 0.000 threading.py:308(_release_save)
    2  0.000  0.000  0.000  0.000 0.000 traitlets.py:629(get)
    1  0.000  0.000  0.000  0.000 0.000 {built-in method
_contextvars.copy_context}
    1  0.000  0.000  0.000  0.000 0.000 threading.py:302(__exit__)
    2  0.000  0.000  0.000  0.000 0.000 {method '__exit__' of
'_thread.lock' objects}
    2  0.000  0.000  0.000  0.000 0.000
traitlets.py:3486(validate_elements)
    1  0.000  0.000  0.000  0.000 0.000 {built-in method nt.getpid}
    1  0.000  0.000  0.000  0.000 0.000 threading.py:314(_is_owned)
    1  0.000  0.000  0.000  0.000 0.000 iostream.py:216(_check_mp_mode)
    4  0.000  0.000  0.000  0.000 0.000 {method 'get' of 'dict' objects}
    3  0.000  0.000  0.000  0.000 0.000 {method 'append' of
'collections.deque' objects}
    1  0.000  0.000  0.000  0.000 0.000
selector_events.py:746(_process_events)
    2  0.000  0.000  0.000  0.000 0.000 {method 'acquire' of

```



```

'_thread.lock' objects}
      1      0.000      0.000      0.000      0.000 {method '__enter__' of
'_thread.RLock' objects}
      14      0.000      0.000      0.000      0.000 typing.py:2119(cast)
      2      0.000      0.000      0.000      0.000 base_events.py:1989(get_debug)
      1      0.000      0.000      0.000      0.000 {method '__enter__' of
'_thread.lock' objects}
      1      0.000      0.000      0.000      0.000 iostream.py:255(closed)
      1      0.000      0.000      0.000      0.000 base_events.py:537(_check_closed)
      2      0.000      0.000      0.000      0.000 {method 'extend' of 'list'
objects}
      1      0.000      0.000      0.000      0.000 {built-in method time.monotonic}
      1      0.000      0.000      0.000      0.000 {method 'cancelled' of
'_asyncio.Future' objects}
      1      0.000      0.000      0.000      0.000 base_events.py:717(is_closed)
      1      0.000      0.000      0.000      0.000 queues.py:59(_set_timeout)
      1      0.000      0.000      0.000      0.000 {method 'release' of
'_thread.lock' objects}
      1      0.000      0.000      0.000      0.000 {method '_is_owned' of
'_thread.RLock' objects}
      1      0.000      0.000      0.000      0.000 {method '__exit__' of
'_thread.RLock' objects}

```

5 Tổng kết

- Độ phức tạp (complexity) cho biết lượng thời gian hoặc bộ nhớ mà thuật toán cần khi dữ liệu tăng.
- Ký hiệu Big-O giúp mô tả tốc độ tăng của chi phí xử lý theo kích thước đầu vào n .
- Khi ước lượng Big-O, ta thường dựa vào cấu trúc mã như phép toán đơn, vòng lặp đơn, vòng lặp lồng nhau và vòng lặp giảm một nửa mỗi bước.
- Trong phân tích độ phức tạp, ta quan tâm đến xu hướng tăng khi n lớn, nên thường bỏ qua hằng số và chỉ giữ bậc lớn nhất.
- Các magic commands như `%time`, `%timeit`, `%%time`, `%%timeit`, `%prun`, `%%prun` giúp kiểm tra hiệu năng thực tế ngay trong Jupyter Notebook.
- Cần kết hợp cả phân tích lý thuyết (Big-O) và đo đạc thực nghiệm (profiling) để đánh giá chương trình chính xác hơn.

Checklist tự kiểm tra: - Bạn đã phân biệt được $O(1)$, $O(n)$, $O(n^2)$ và $O(\log n)$ chưa? - Bạn đã biết cách ước lượng Big-O từ cấu trúc vòng lặp trong mã chưa? - Bạn đã biết khi nào dùng `%timeit`, `%prun` chưa? - Bạn có thể giải thích vì sao hai chương trình cho cùng kết quả nhưng có thể khác nhau rất nhiều về thời gian chạy không?

6 Bài tập (Problems)

6.1 Bài 1: Nhận diện độ phức tạp

Xác định Big-O cho các đoạn mã sau và giải thích ngắn gọn cách suy luận: - Một phép truy cập phần tử trong danh sách. - Một vòng lặp duyệt toàn bộ danh sách. - Hai vòng lặp lồng nhau cùng

duyệt danh sách. - Một vòng lặp `while` mà mỗi bước chia đôi giá trị của `n`.

6.2 Bài 2: Tính Big-O từ mã nguồn

Viết 4 hàm minh họa cho các mức độ phức tạp sau: - $O(1)$ - $O(n)$ - $O(n^2)$ - $O(\log n)$

Yêu cầu: - Ghi chú Big-O ngay trong từng hàm. - Chạy thử với dữ liệu nhỏ để kiểm tra kết quả.

6.3 Bài 3: So sánh bằng magic command

Viết 2 cách tính tổng bình phương các số từ 0 đến `n-1`: - Cách 1: dùng vòng lặp `for`. - Cách 2: dùng biểu thức sinh (generator expression) hoặc công thức toán học nếu có.

Yêu cầu: - Dự đoán cách nào nhanh hơn. - Dùng `%time` hoặc `%timeit` để kiểm tra. - Giải thích vì sao thời gian chạy khác nhau.

6.4 Bài 4: Dùng Python profiler trong notebook

Chọn một hàm có vòng lặp trong bài và đo bằng `%prun` hoặc `%%prun`.

Yêu cầu: - Quan sát hàm nào tốn nhiều thời gian nhất. - Viết 1 nhận xét ngắn về điểm nghẽn (bottleneck).

6.5 Bài 5: So sánh thời gian khi dùng và không dùng list comprehension

Viết 2 cách tạo danh sách bình phương của các số từ 0 đến `n-1`: - Cách 1: dùng vòng lặp `for` kết hợp `append()`. - Cách 2: dùng list comprehension.

Yêu cầu: - Dự đoán cách nào nhanh hơn. - Dùng `%time` hoặc `%timeit` để đo thời gian chạy của hai cách. - So sánh kết quả và nêu nhận xét. - Giải thích vì sao list comprehension thường ngắn gọn và có thể chạy nhanh hơn trong Python.

6.6 Bài 6: Vận dụng thực tế

Giả sử bạn cần xử lý một tệp dữ liệu rất lớn: - Chỉ ra 2 đoạn mã có thể trở thành điểm nghẽn. - Nêu cách bạn ước lượng Big-O trước khi chạy chương trình. - Chọn một trong các công cụ `%timeit`, `%prun`, `%lprun` để kiểm tra và giải thích lý do chọn.